西安交通大学
XI'AN JIAOTONG UNIVERSITY

CSCC 全国大学生计算机系统能力大赛

基于GPU的服务资源卸载技术探索

GPUC——面向透明存储压缩的GPU解压卸载原型

2026 全国大学生计算机系统能力大赛 — OS 功能挑战赛道

西安交通大学 · 队伍 **404notfound**

成员：朱嘉乐（队长）、贾茹、边程曦

指导老师：聂世强、刘松

用户态原型已跑通，验证材料可复核

透明访问

LD_PRELOAD Hook 接入常用 POSIX I/O，维护逻辑长度、偏移与读取语义。

压缩与解压路径

前台先写 RAW，后台压缩；SGLZ 与 LZ4 支持 CPU/GPU 后端切换。

工程处理

长期 GPU 工作进程、CRC/SHA 校验、原子替换与 CPU 回退。

现有交付与验证材料

- **15 个源文件 · 7902 行:**

src/ 与 include/，含 4 个测试入口源文件

- **89 个测试与实验文件:**

含 68 个 Shell 脚本、辅助程序与 Vdbench 配置

- **两台整机分阶段实测:**

V100 与 RTX 4090；结果不作显卡单变量比较

已完成

核心读写与压缩/解压链路

已验证

正确性、代表性性能、资源、稳定与软件故障

当前边界

用户态动态链接原型；掉电与 GPU reset 未覆盖

CSCC 全国大学生计算机系统能力大赛

目录

CONTENTS

01 问题与背景

02 解决方案

03 创新设计

04 实验结果

05 团队与总结

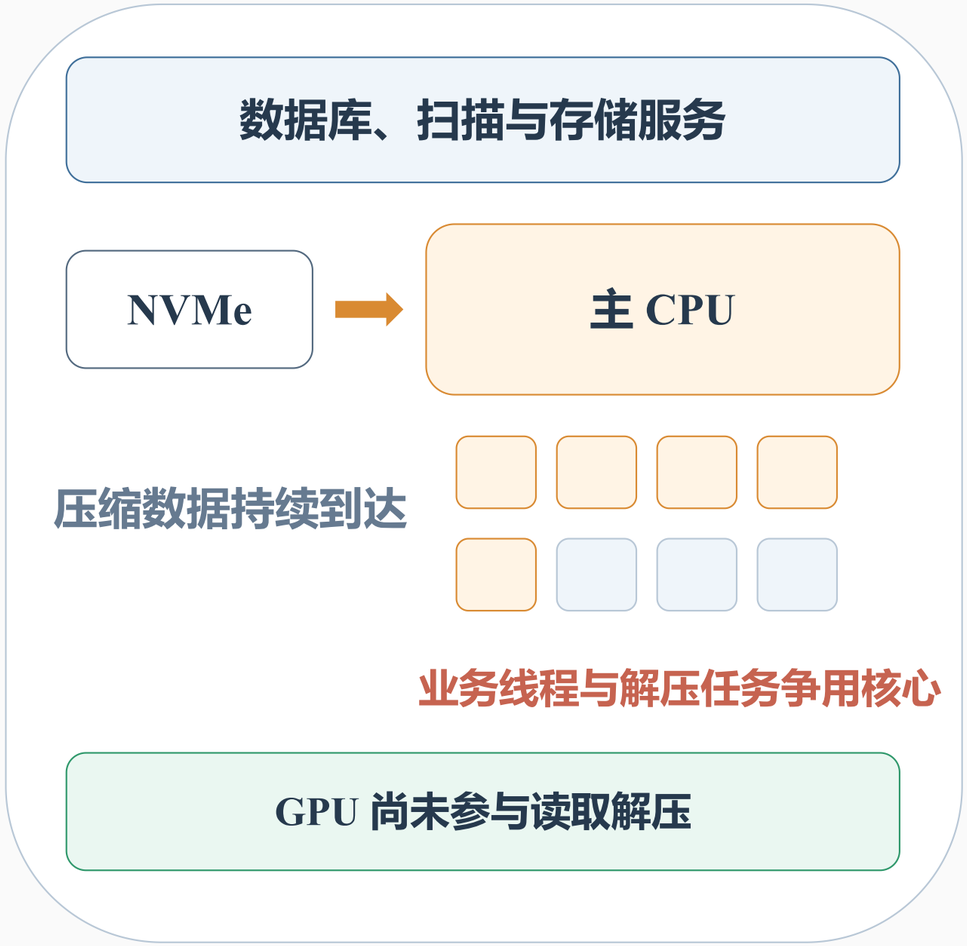
01

问题与背景

为什么存储压缩会形成 CPU 资源问题？本项目为什么先选择用户态入口？

01 问题与背景——压缩节省存储，但解压会占用业务 CPU

真实场景：数据库与存储服务共机运行



压缩的交换

减少空间与传输量，但读取时增加解压计算。

瓶颈的转移

NVMe 与页缓存变快后，解压更容易进入关键路径。

共机的争用

CPU 解压与业务争夺核心，并增加主机内存访问。

Song 等[1]指出：压缩进入读写关键路径后会阻塞用户 I/O，因此应尽量与前台请求解耦。

Peñaranda 等[2]指出：GPU 更适合较大数据，小任务可能由 CPU 处理更快，因此需要按任务规模选择后端。

核心问题：在相同解压工作量下，GPU 能否减少主 CPU 占用，让业务获得更多 CPU 资源？

01 问题与背景——现有压缩路线各有取舍，本项目先验证机制可行性

背景判断：存储压缩可以落在不同系统层次，透明范围、部署成本和工程复杂度并不相同。

三种系统落点及论文依据

Device Mapper / 块层

- 对 ext4/XFS 较透明，但需自行处理块映射和一致性恢复。
- Tan 与 Zhu[3]说明 Device Mapper 可以通过逻辑—物理映射向上提供标准块设备。

Btrfs / OpenZFS 文件系统层

- 原生集成 CoW、校验、快照和透明压缩，但使用时绑定文件系统。
- Dakic 等[4]指出，文件系统选择取决于具体数据访问需求。

用户态机制原型

- 无需修改内核，适合快速验证机制，但透明范围有限。
- Cho 等[5]指出，用户态开发更灵活，但内核—用户态通信会产生额外开销。

项目定位：本项目选择用户态入口回答“机制是否可行、在什么条件下有收益”，不把原型表述为已经完成的生产级 DM target 或文件系统压缩实现。

02

解决方案

从整体架构、透明语义、文件格式、计算后端和读写路径说明原型如何工作。

02 解决方案——系统把应用访问、后台压缩和读取解压分开

业务继续使用原有 POSIX 接口，写入、后台压缩与读取解压彼此分离

应用层

- 数据库（SQLite）
- 存储测试（fio）
- 并发测试（Vdbench）
- 普通 POSIX 应用

↓

透明访问层

- LD_PRELOAD Hook：接管常用文件访问，维护逻辑长度、偏移与读取语义

↓

核心处理层

1. 写入：应用写入 → 写入 RAW 文件 → 保留逻辑内容 → 立即返回
2. 后台压缩：后台扫描 → 批量压缩 → 生成临时文件 → 原子替换
3. 透明读取：识别压缩对象 → CPU 或 GPU 解压 → 校验与回退 → 返回数据

↓

资源与数据层

- NVMe 与页缓存
- 主机内存
- GPU 显存与工作进程

结论：业务继续使用原有 POSIX 接口；压缩发生时间与解压计算后端可以独立调整。

02 解决方案——透明语义让应用看到原始内容和逻辑大小

物理文件可以压缩，但应用仍按原始文件解释长度、偏移和读取内容

已接入的常用接口

open	read / pread	fstat / lseek	mmap

应用可见能力	普通文件	GPUC 压缩对象
访问方式	POSIX 接口	POSIX 接口
文件大小	物理大小	原始逻辑大小
读取结果	原始内容	解压后的原始内容
破坏性写入	允许	返回 EROFS

Hook 维护的信息

- 对象格式与文件描述符状态
- 逻辑偏移、逻辑长度与只读状态

对象状态对应用一致

- RAW
- COMPRESSED
- INLINE

结论：三种对象都向应用返回相同的逻辑内容和长度。

02 解决方案——自描述文件格式支持随机访问、校验和安全替换

文件组织

56 B 文件头　　**24 B / 块索引**

4 KiB 对齐的数据区

格式识别

- 压缩算法标识

逻辑语义

- 原始逻辑长度

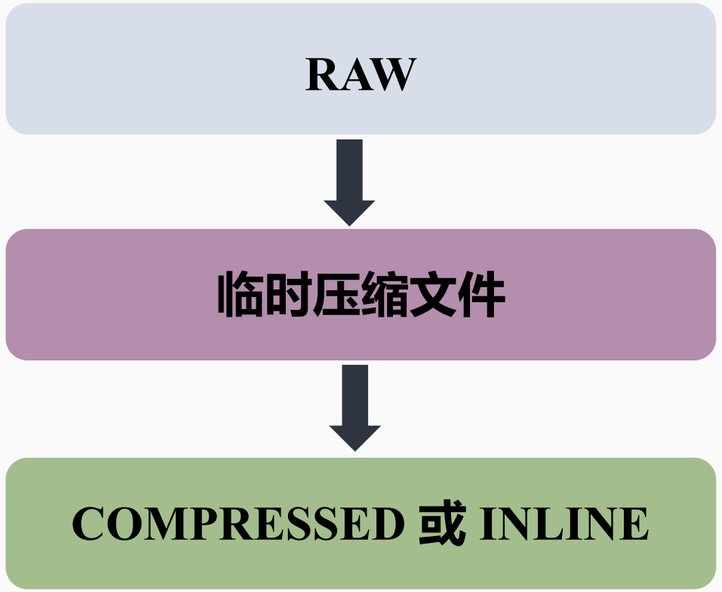
算法选择

- 压缩算法标识

随机访问

- 物理偏移与 CRC32

对象状态变化



生成临时文件 → 写完整内容 → fsync → rename；
收益不足或处理失败时保留 RAW。

结论：读取前检查格式、边界与校验；压缩失败不会覆盖仍然有效的 RAW 文件。

02 解决方案——统一后端接口支持算法切换与 CPU 回退

输入：压缩块批次、算法类型和逻辑范围　→　输出：逻辑数据与 CRC 校验结果

算法	CPU路径	GPU路径
SGLZ	SGLZ CPU	SGLZ CUDA
LZ4	liblz4	nvCOMP LZ4

统一调度与回退

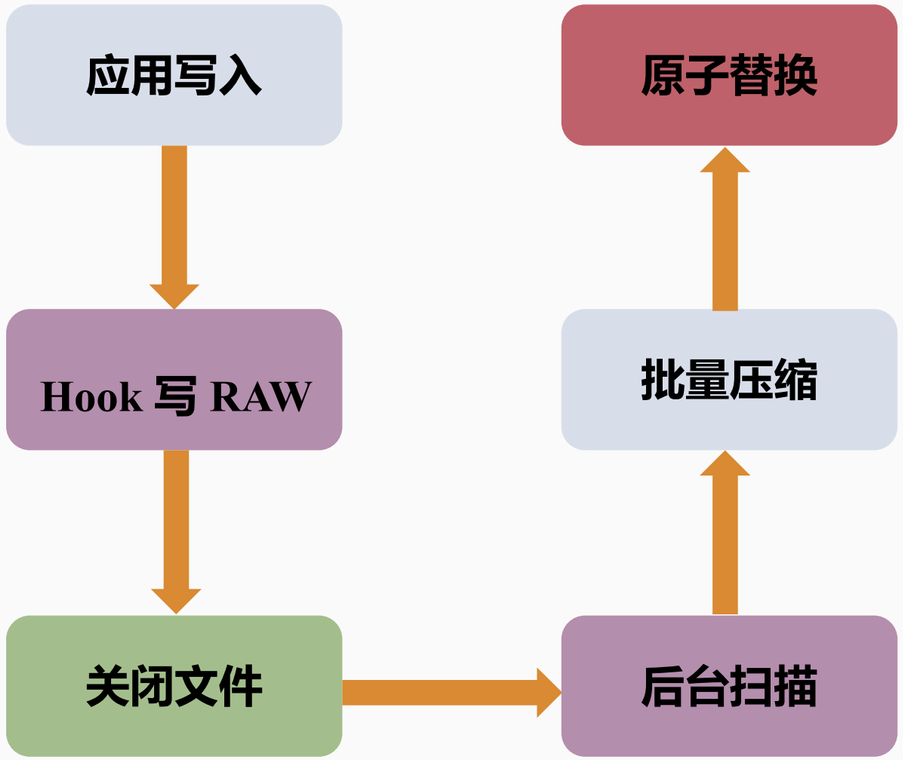
- 支持单块与批处理
- 小请求可选择 CPU
- GPU 初始化失败或设备不可见时回退 CPU
- 上层调用方式保持一致

识别算法 → **选择 CPU 或 GPU** → **校验结果** → **必要时回退 CPU**

结论：项目不是只接入一个 CUDA 内核，而是把多算法、多后端与回退统一到同一读取语义下。

02 解决方案——后台压缩避免把压缩延迟直接加到前台写入

写入与压缩流程



- **前台快速返回:**业务写入不等待同步压缩。
- **后台选择时机:**Daemon 扫描并批量处理。
- **收益不足保留 RAW:**避免为了压缩而扩大文件。
- **失败不替换:**错误时仍保留有效原文件。

代价：RAW 在后台处理完成前会暂时占用额外空间，压缩时机也需要策略控制。

结论：前台与后台解耦降低写入等待；原子替换保证失败时仍有有效数据可读。

02 解决方案——读取路径同时处理索引、传输、解压和校验

解析逻辑范围 → 定位块索引 → 批量读取 → 传输至 GPU → 解压并回传 → CRC 校验 → 返回应用

长期 GPU 工作进程复用资源

- CUDA 上下文
- 数据流与事件
- 固定页内存与复用缓冲区

读取调度处理

- 合并逻辑范围内的块请求
- 自适应预读减少碎片化调用
- 小请求或 GPU 不可用时走 CPU
- 校验通过后才返回应用

真实成本 = I/O + 数据传输 + 任务调度 + 解压计算，而不是单个内核时间。

结论：长期工作进程与批处理用于摊薄重复初始化、调用和传输的固定开销。

02 解决方案——正确性与故障处理覆盖正常链路之外的工程问题

01

数 据 正 确 性

- 块级 CRC 检查
- 测试前后 SHA 对比
- 格式、偏移与长度边界检查

02

并发与生命周期

- 文件描述符引用计数
- 共享状态加锁
- 资源在锁外销毁

03

兼 容 处 理

- mmap 使用 userfaultfd 路径
- 环境不支持时同步回退

04

故 障 处 理

- GPU 不可见时回退 CPU
- ENOSPC、SIGKILL、中途重启不替换 RAW
- rename 前异常保留原文件

结论：性能路径可以失败或回退，但数据内容和原文件状态必须保持可控。

03

创新设计

概括项目在机制组合、资源卸载和验证方法上的主要贡献。

03 创新设计——项目贡献是透明接入、资源卸载与证据闭环

架构解耦与验证闭环构成项目核心贡献

访问与计算解耦

- 应用**零改动**接入，Hook 保持逻辑内容与大小不变
- 解压后端可无缝切换 **CPU/GPU** 或 **SGLZ/nvCOMP LZ4**

前台与后台解耦

- 前台**先写 RAW**，业务写入无阻塞
- Daemon 后台 **Batch 压缩** 并原子替换
- 压缩开销与业务写入**解耦隔离**

调度与传输协同

- **长期 GPU Worker** 持续驻留降低启动开销
- **Pinned Memory** 锁定页加速 H2D 传输
- **Batch 与自适应预读** 三重机制降低固定开销

验证闭环：多维度联合验证，而非单项吞吐报告

04

实验结果

按实验设计、共机性能、兼容性、稳定性和资源卸载依次给出证据。

04 实验结果——实验按正确性、性能与资源占用三条线展开

正确性

数据完整性、解压缩一致性、端到端无误码

性能

I/O 带宽、时延、并发吞吐、业务 QPS

资源占用

CPU 利用率、DRAM 带宽、GPU 状态与系统开销

目的	工具或指标
I/O 与时延	fio：带宽、IOPS、平均时延与尾时延
并发读取	Vdbench：顺序/随机、多线程、混合负载
业务影响	SQLite：QPS 与 p99 延迟
CPU 与系统资源	pidstat、sar、perf
DRAM 带宽	Intel PCM
GPU 状态	nvidia-smi 与项目内部计数
数据正确性	SHA 与 error 计数

测试平台

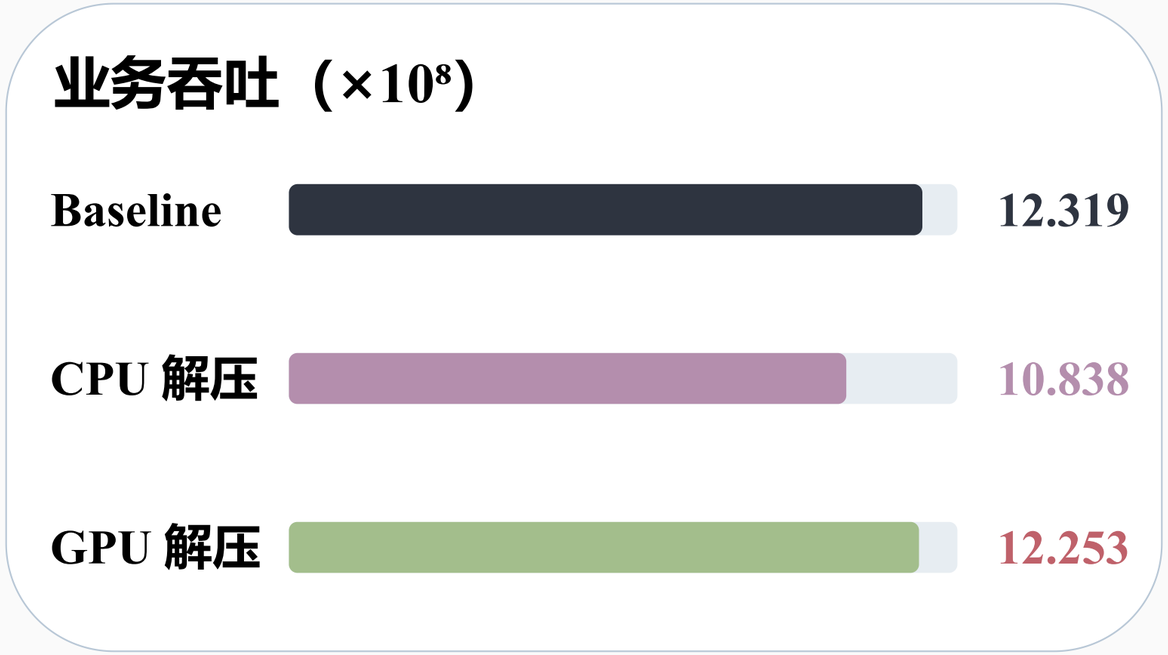
V100（sm_70）与 RTX 4090（sm_89），分属不同整机，用于**分阶段验证**，不做 GPU 型号单变量比较

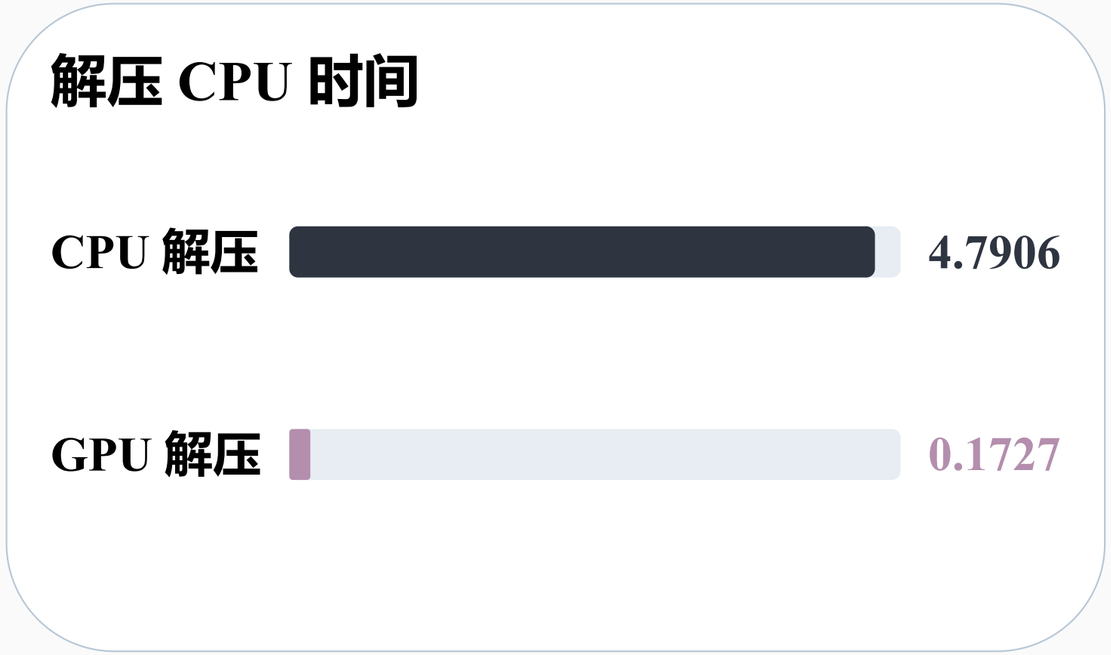
统计方法

短测执行**5 次**，报告均值与**95% 置信区间**；稳定性实验连续运行**1 小时**。所有结论可追溯至 results/final/ 中的日志、CSV 或汇总文件

04 实验结果——约 4 GiB/s 解压负载下，GPU 基本恢复业务吞吐并显著减少 CPU 时间

条件：4 个 CPU 业务线程，与约 4096 MiB/s 的 LZ4 解压负载共机





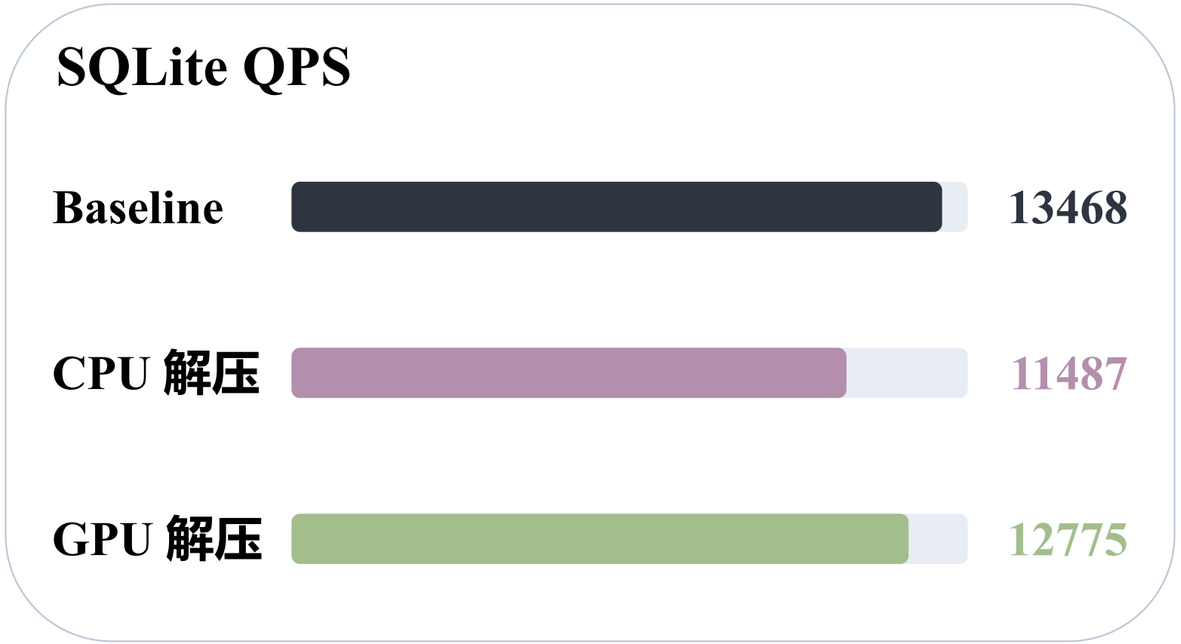
+13.05%
GPU 相对 CPU 的业务吞吐

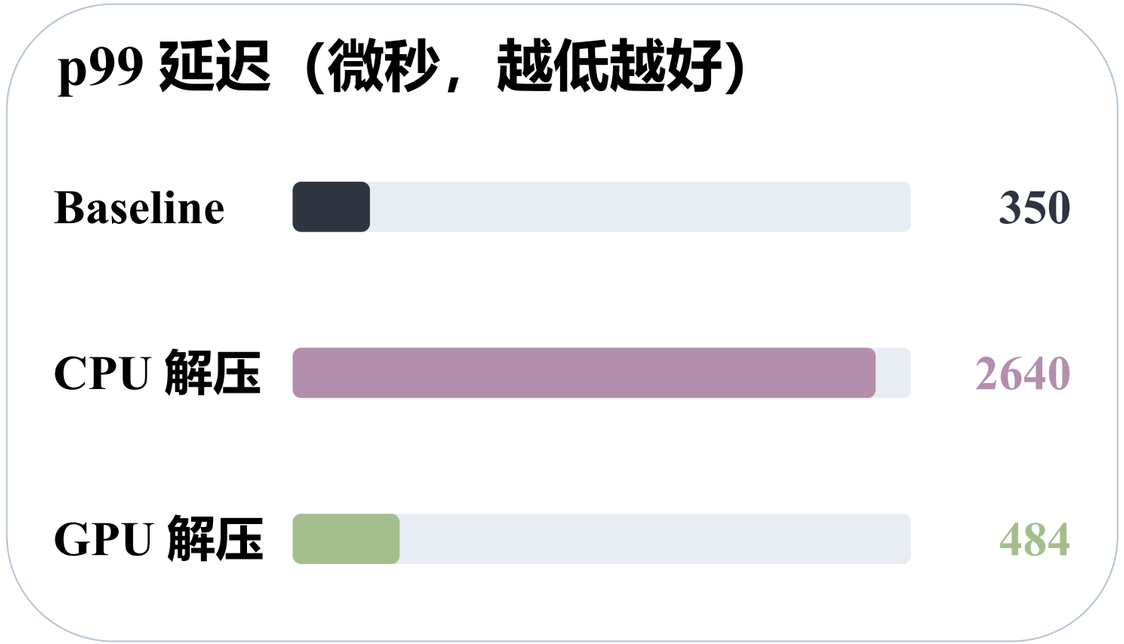
-96.4%
GPU 相对 CPU 的解压时间

-0.54%
GPU 与 Baseline 的吞吐差距

该场景中 GPU 的主要价值是降低解压与业务之间的 CPU 争用，而不是让存储介质变快

04 实验结果——SQLite 实验显示 GPU 解压降低共机干扰，但未超过无解压基线





+11.21%
QPS：GPU 相对 CPU 解压

-81.66%
p99：GPU 相对 CPU 解压

-5.14%
GPU QPS 相对 Baseline

GPU 没有加速 SQLite 本身，而是减轻了共机解压对 SQLite 的干扰。

04 实验结果——fio 与 Vdbench 证明接口可用和数据正确，也暴露小随机读边界

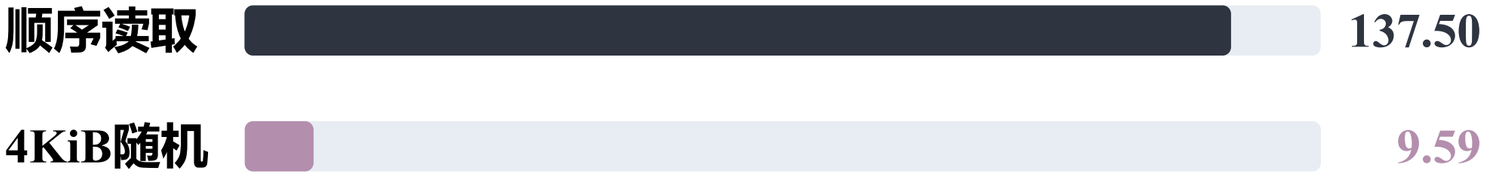
fio 正式测试

- 覆盖 RAW、COMPRESSED、INLINE
- 顺序读、随机读与 mmap
- runtime=60、ramp_time=60
- error=0、无超时、SHA 一致

Vdbench 正式测试

- 顺序读、随机读与 50/50 混合负载
- 4 线程、5 次重复
- 覆盖 Baseline、RAW 与压缩路径
- 读取结果通过 SHA 校验

LZ4 代表读取带宽（MiB/s）



性能边界

两者均低于普通文件路径；小随机读的固定开销尤其明显。

正式工具验证了接口兼容性与数据正确性，但当前原型并非所有 I/O 模式都能获益。

04 实验结果——一小时稳定读取无错误，确定性软件故障能够保留或恢复有效数据

3600 s	473.79 MiB/s	error=0	SHA 一致
实际运行时间	平均吞吐	读取错误	前后内容

已执行的软件故障测试

- ENOSPC
- 压缩进程 SIGKILL
- 临时文件残留后重启
- GPU 不可见时 CPU 回退
- /dev/full 场景 5/5 恢复成功

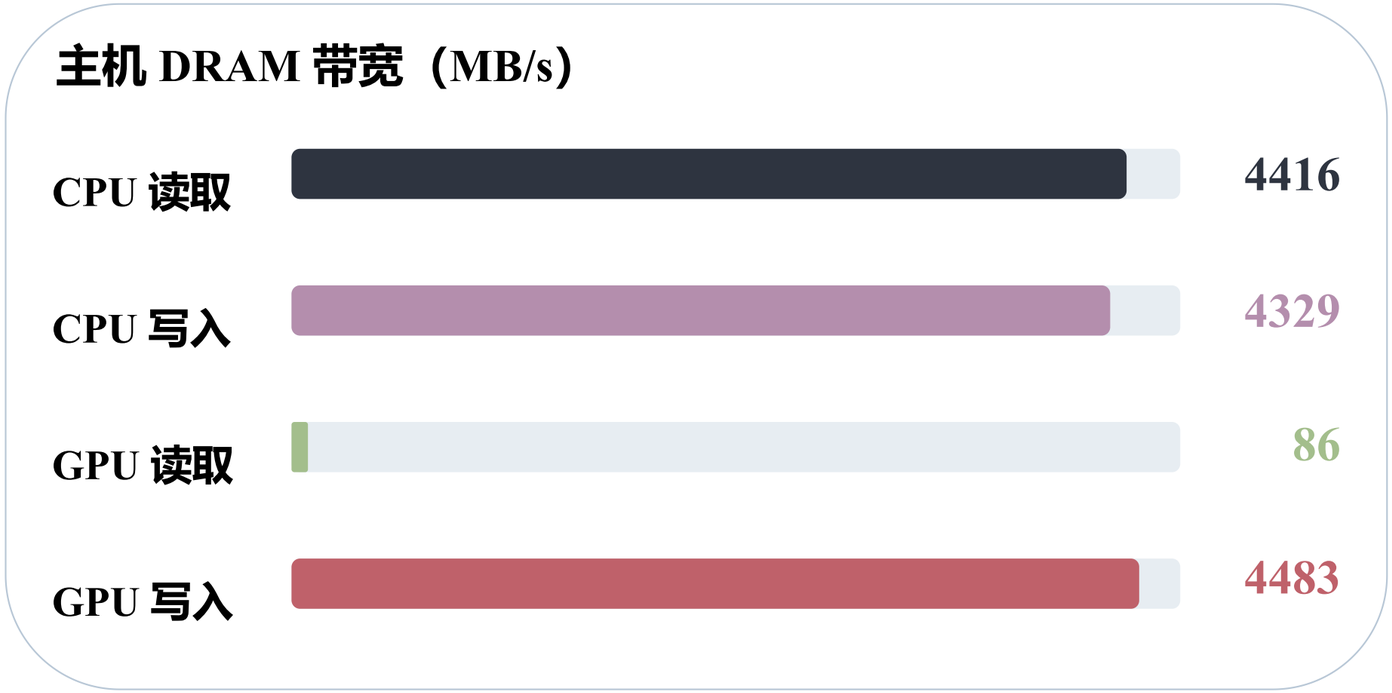
本次实验的明确边界

- 数据量：512 MiB
- 块大小：128 KiB
- 并发：4 个任务
- 真实掉电未覆盖
- 真实 GPU reset 未覆盖

实验支持“一小时持续读取正确”和“确定性软件故障有效”，不代表完成生产级灾难恢复验证。

04 实验结果——同吞吐高可压负载中，GPU 路径降低约 47.75% 主机 DRAM 带宽

条件：V100 平台；CPU 与 GPU 路径均维持约 4093 MiB/s 的解压吞吐



8745.46

CPU 路径合计 MB/s

4569.51

GPU 路径合计 MB/s

原因解释

GPU 路径减少主机侧压缩输入读取，但解压结果仍需写回主机内存。

-47.75%

GPU 相对 CPU 的总带宽

该结果表示特定负载下的内存带宽下降，不等同于 RSS 或内存容量下降。

05

团队与总结

团队分工、代码来源边界、项目结论和后续演进方向。

05 团队与总结——三名成员共同完成系统、算法与实验闭环

成员	主要职责
朱嘉乐（队长）	系统架构；Hook、Daemon、SGLZ/nvCOMP 后端集成、项目整合
贾茹	存储与 I/O 优化；io_uring、内存池、预读、mmap；测试与性能评测
边程曦	文档、PPT、演示视频与提交材料整理

① 格式与 Hook

AW/COMPRESSED 格式、POSIX Hook 链路

② 后端接入

CPU/GPU 双解压后端集成

③ Worker 与调度

期 Worker、Batch、校验与回退

④ 分阶段实验

100 可行性、RTX 4090 适用边界

⑤ 汇总答辩

复核边界、形成结论、输出材料

五阶段闭环：从 RAW 格式到答辩材料，完整交付。

05 团队与总结——原创实现、第三方依赖与 AI 辅助边界清晰

核心方案选择、实验执行、数据复核均由团队成员独立完成

AI 辅助仅限测试脚本草拟、文档整理、结果表格生成和代码审阅建议

原创实现

- 透明 I/O Hook
- 文件格式与逻辑语义
- 后台压缩 Daemon
- SGLZ CUDA 路径与统一后端接口
- 长期 GPU Worker
- Batch、回退与原子替换
- 测试脚本与结果汇

第三方依赖

- CUDA Runtime
- nvCOMP
- liblz4
- fio
- Vdbench
- SQLite
- PCM

AI 辅助范围

- 透明 I/O Hook
- 测试脚本草拟
- 文档整理与表格生成
- 代码审阅建议
- 方案、集成、实验与结论由团队确认

05 团队与总结——论文与技术资料

[1] Song 等，Breathing New Life into Compression，Journal of Systems Architecture，2025。doi:10.1016/j.sysarc.2025.103432

[2] Peñaranda 等，Hybrid-Smash: A Heterogeneous CPU-GPU Compression Library，IEEE Access，2024。doi:10.1109/ACCESS.2024.3371253

[3] Tan 与 Zhu，OCM: An Overcapacity Mapping-Based Virtual Disk Approach，Electronics，2025。doi:10.3390/electronics14204091

[4] Dakic 等，Btrfs and ZFS Performance for Different Use Cases，Computers，2024。doi:10.3390/computers13060139

[5] Cho 等，RFUSE: Modernizing Userspace Filesystem Framework，FAST 2024。usenix.org/conference/fast24/presentation/cho

补充资料：Linux Device Mapper、Btrfs、OpenZFS、FUSE/ublk、nvCOMP 与 CUDA Runtime 官方文档。

西安交通大学
XI'AN JIAOTONG UNIVERSITY

CSCC 全国大学生计算机系统能力大赛

谢谢各位老师，敬请批评指正！

2026 全国大学生计算机系统能力大赛 — OS 功能挑战赛道

西安交通大学 · 队伍 **404notfound**

成员：朱嘉乐（队长）、贾茹、边程曦　　指导老师：聂世强、刘松